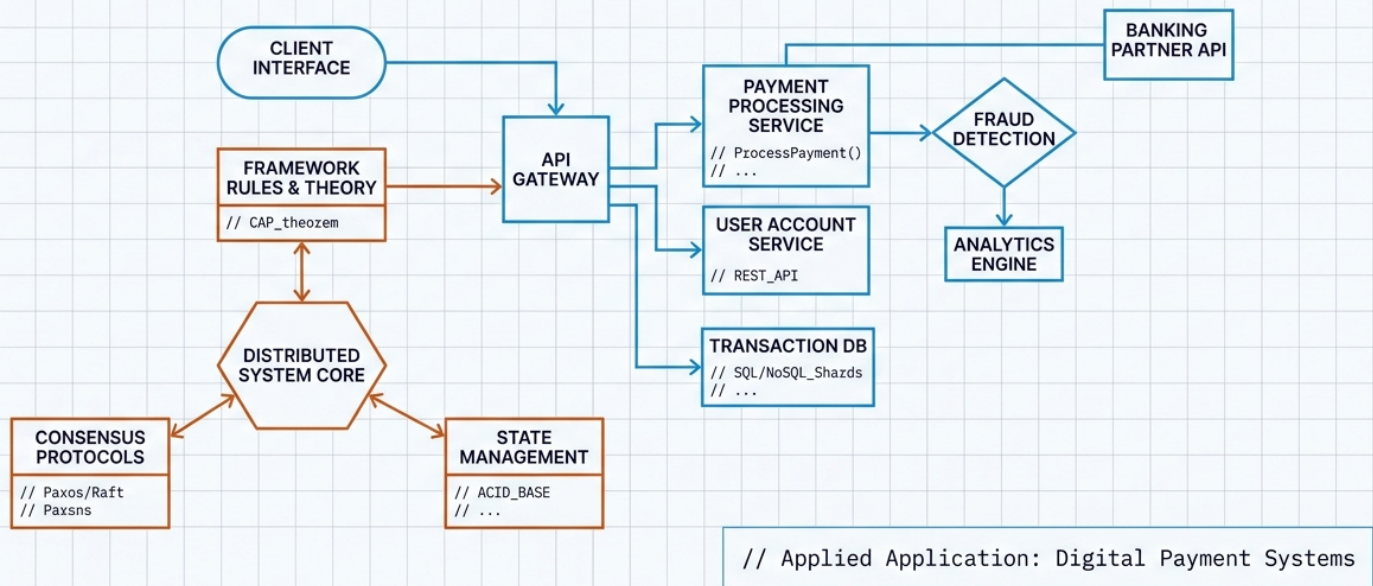
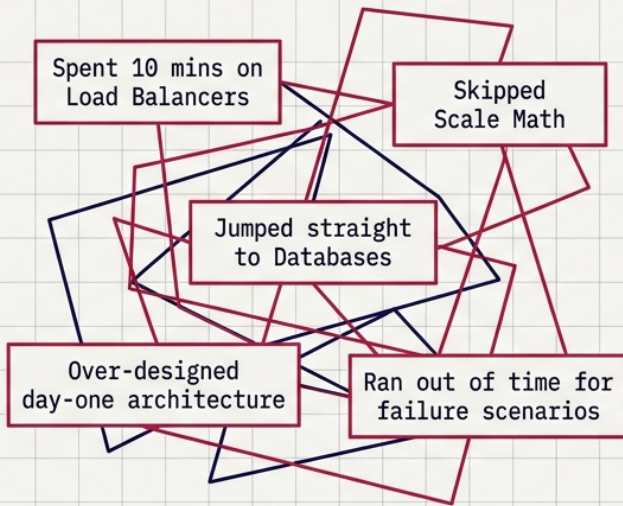


System Design Master Framework: The 45-Minute Playbook



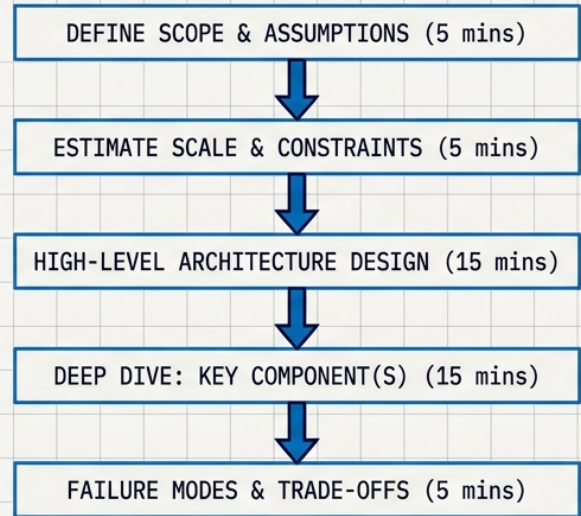
The Unstructured Interview Trap

The 45-Minute Chaos



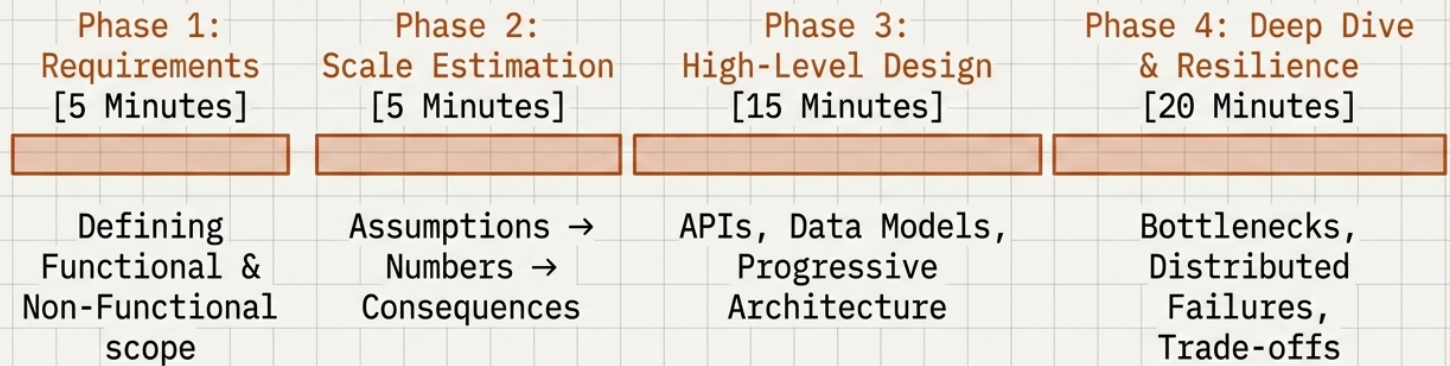
Evaluating pure technical trivia

The Structured Architect



Evaluating prioritization, assumptions, and trade-offs

The 45-Minute Blueprint



Phase 1: Separating the Signal from the Noise

Functional Requirements (What it does)

- Limit initial scope to 3–4 core user journeys.
- Ask: Who are the users? What are their core actions?

Non-Functional Requirements (How well it performs)

- Focus only on architecture-altering constraints.
- Ask: Latency expectations? Read/write ratio? Geographic distribution?

(Geography alters CDN, replication, and data residency decisions)

**Do not encourage unnecessary questions.
Ask only what changes the architecture.**

Applied: Defining Payment System Boundaries

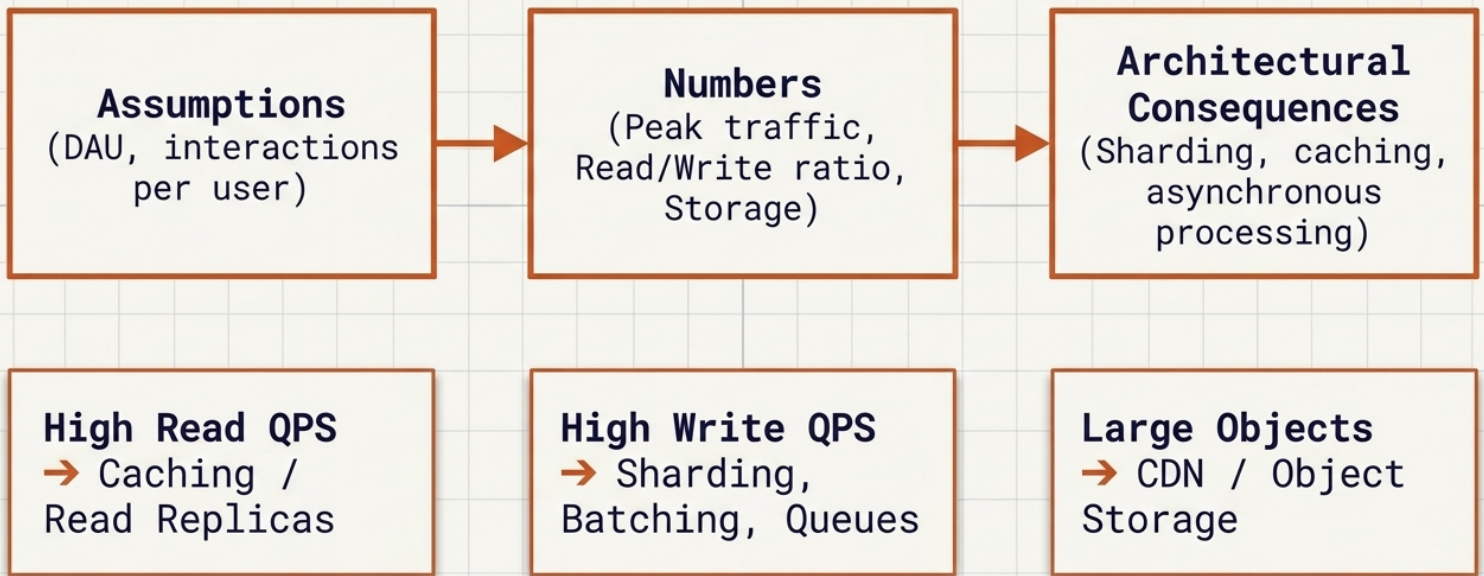
Framework Rule:

Deep Rule:
Stick to architecture-altering constraints.

Payment System Requirements

- | | |
|--|---|
| ✓ Accept payments via PSP (Credit/Debit) | ✓ Integrity & Security (Encryption, consistent state) |
| ✓ User Registration & Auth | ✓ High Availability & Reliability (Redundancy, failovers) |
| ✓ Transaction History (View past records) | ✓ Scalability (Handle peak promotion loads) |
| ✓ Balance Management (Available funds/pending) | ✓ Performance (Strict response times for account updates) |

Phase 2: The Logic of Scale



Applied: Sizing the Payment Engine (Storage & Servers)

Assumptions

50M transactions/day. Peak traffic = 5M tx/hour. Transaction record = 100 bytes. User record = 200 bytes.

Storage Math

$50M \times 100 \text{ bytes} + 200 \text{ bytes} \approx 5 \text{ GB/day}$.

(Consequence: Highly manageable database size, focus on write-speed and durability over massive raw storage)

Server Math

50M Daily Active Users $\approx$ Proxy for 50M RPS total peak capacity.

1 server handles 64,000 RPS.

$50M / 64,000 \approx 782$ servers at peak load.

Applied: Peak Bandwidth Constraints

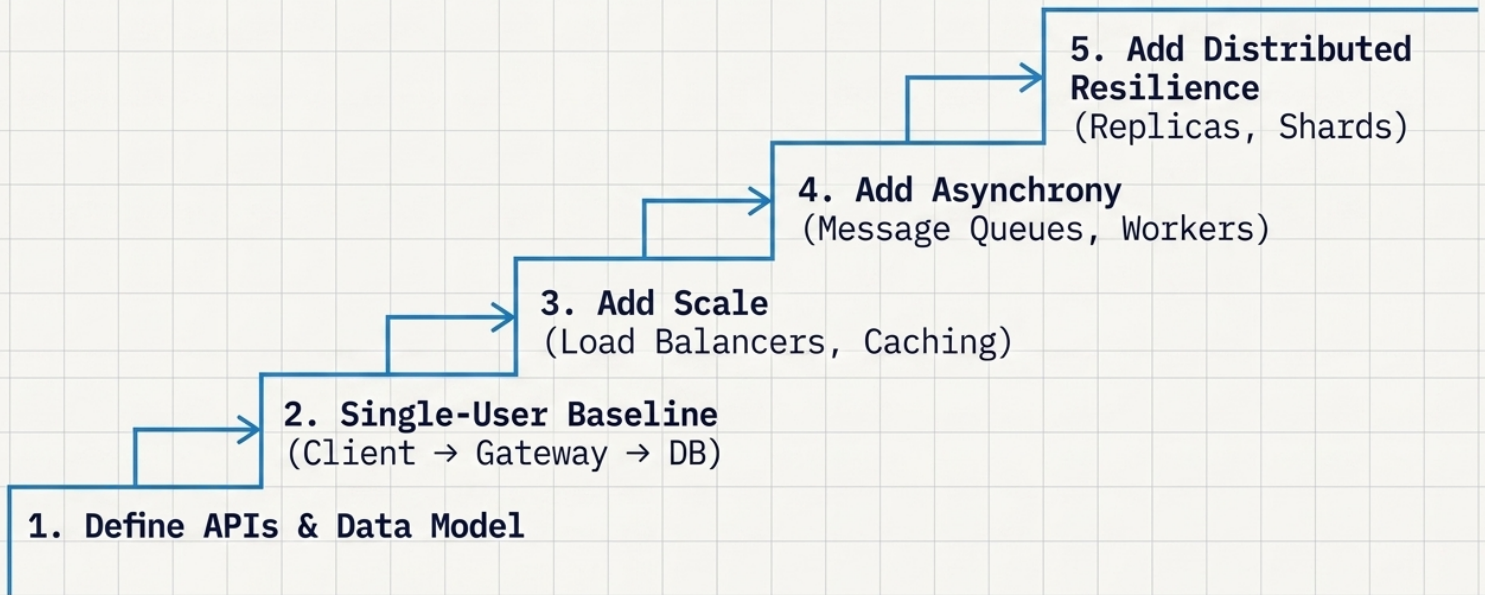


Architectural Consequence

Bandwidth is highly efficient (under 2 Mbps). The system is not network-bound by payload size. The primary constraint will be connection concurrency and latency rather than raw data throughput.

Phase 3: Incremental Architecture

The Rule: Never blindly include every component. Every box must have a justification.



Applied: Defining the Core Payment APIs

```
authenticateUser(username,  
password)
```

```
authorizePayment(amount,  
card_number, exp_date, CVV,  
merchant_id)
```

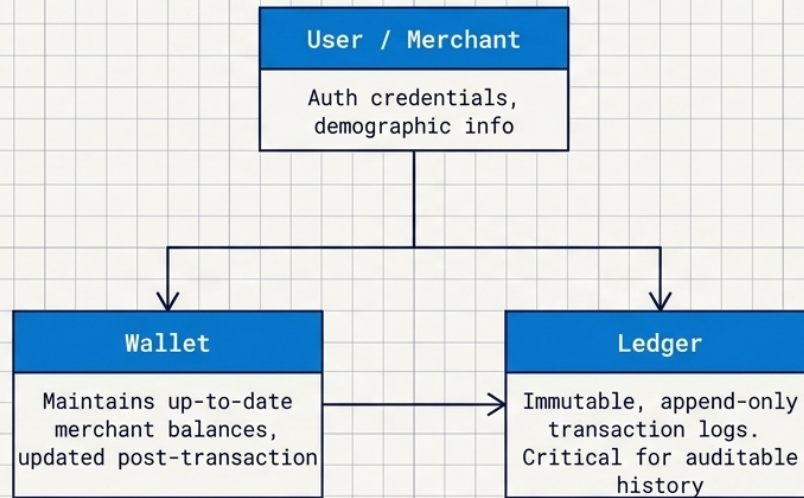
Focus: Verifies funds, reserves amount, returns auth code.

```
capturePayment(authorization_code,  
amount)
```

Focus: Deducts reserved funds, transfers to merchant.

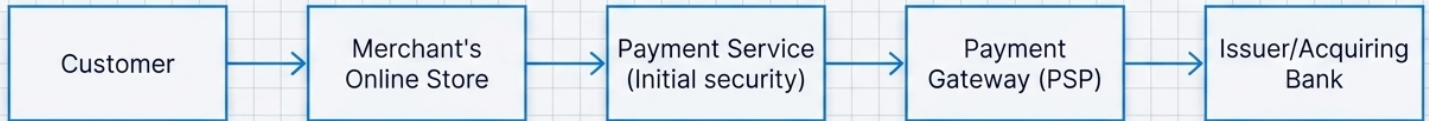
```
getTransactionHistory(customer_id,  
start_date, end_date)
```


Applied: Storage Schema & State Entities



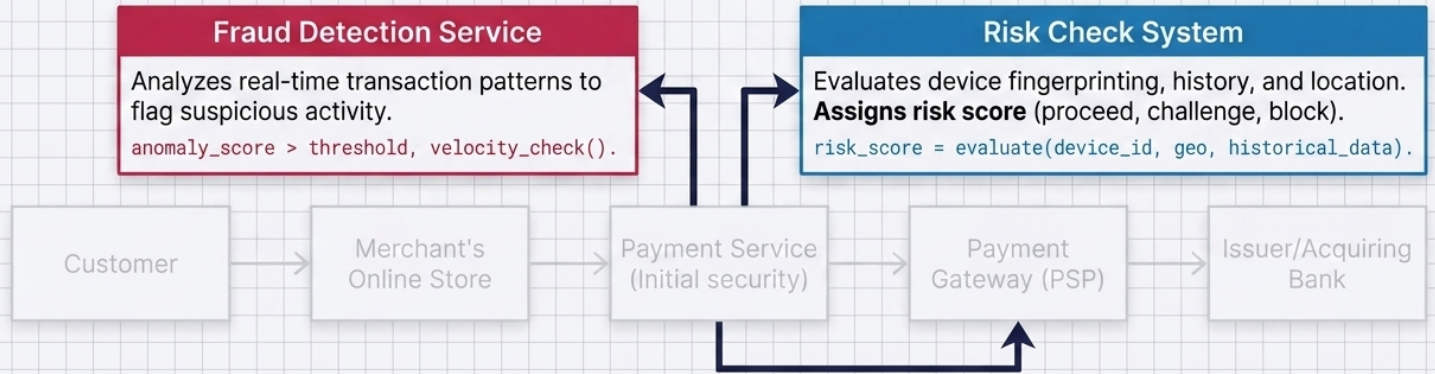
Storage Efficiency Note: User records stored once (200 bytes); transactions reference the user ID.

Applied: The Single-User Baseline



Note: Successful authorization reserves the amount immediately; settlement occurs later in batches.

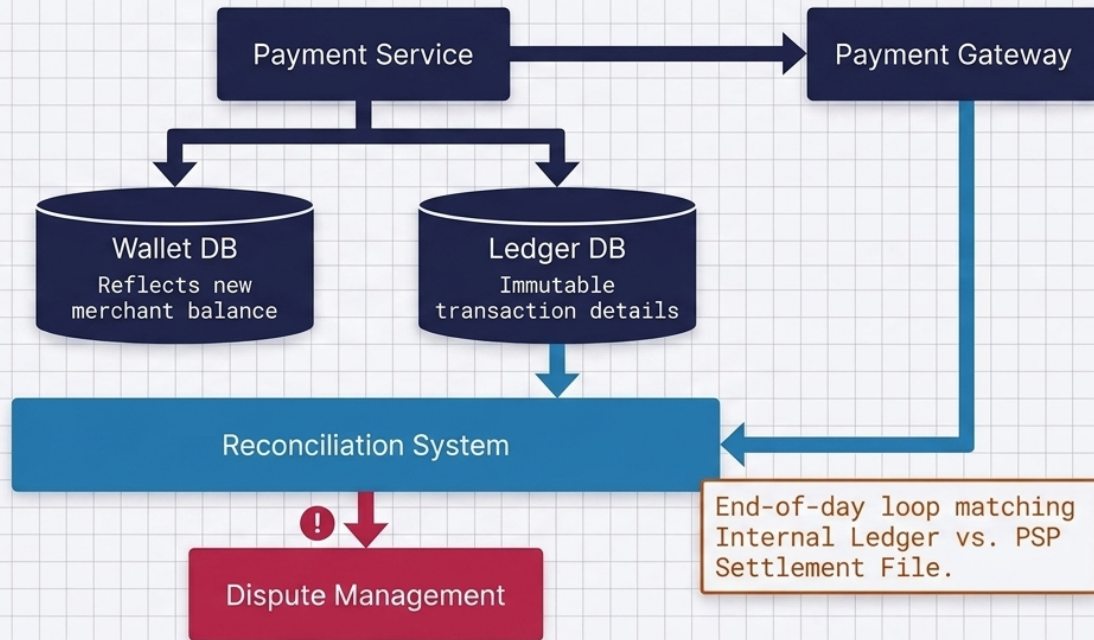
Applied: Decoupling Risk & Fraud



Why Decouple?

Isolating fraud checks from the core gateway allows independent scaling of compute-heavy risk models without bottlenecking transaction throughput.

Applied: State Management & Reconciliation

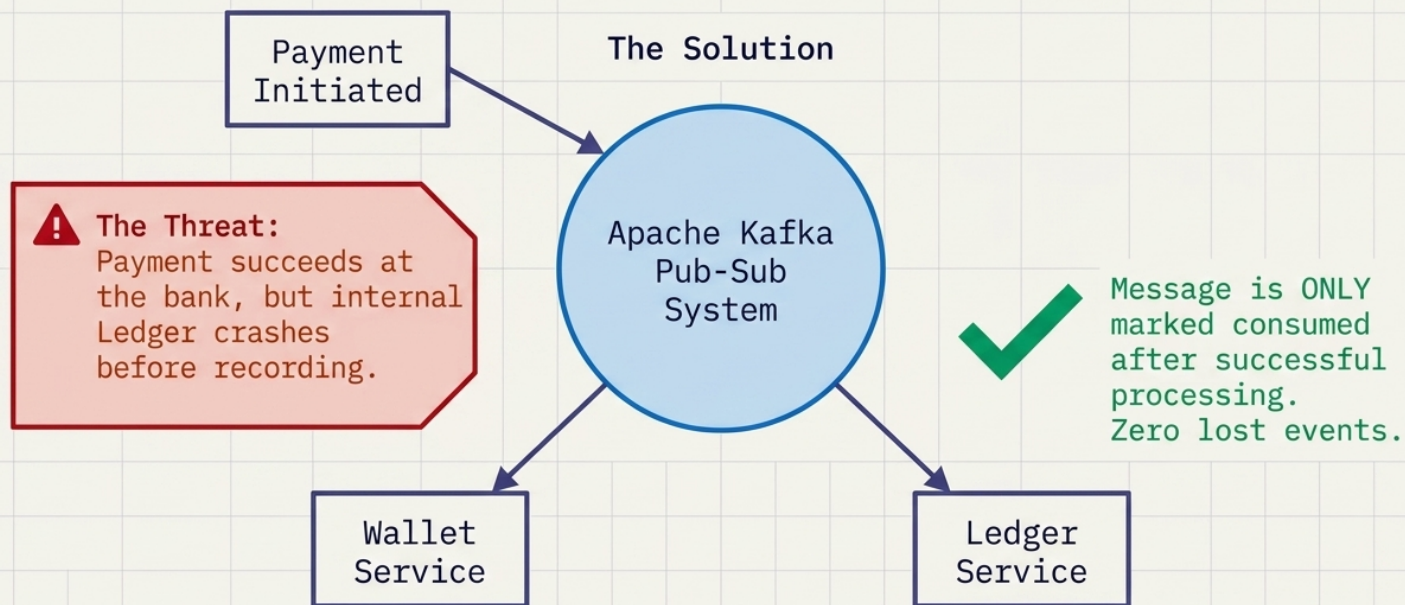


Phase 4: Anticipating the Break

The Rule: The first 25 minutes prove you can build a system. The last 20 minutes prove you can keep it alive.



Applied: Guaranteeing Transaction Completion

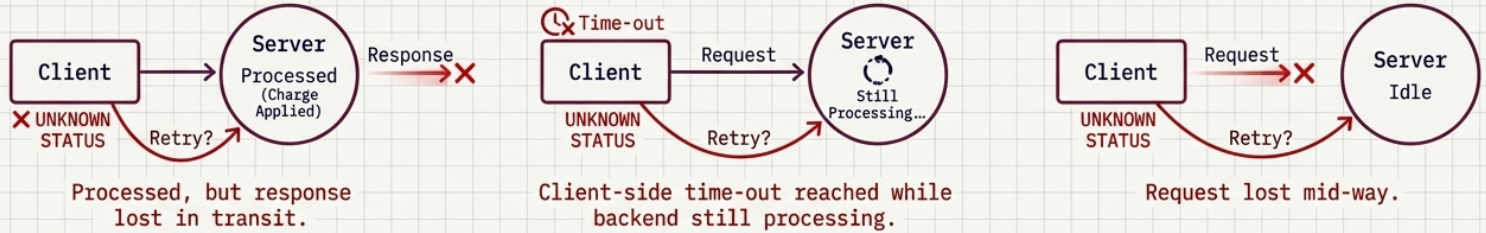


Matrix: Strategies for Transient Failures

	Retries	Time-outs	Fallbacks
Trigger	Network glitches.	Unresponsive services.	Secondary service failure.
Mechanism	Exponential backoff + max limits.	Terminates operation to free resources.	Executes backup process.
Payment Use Case	Re-attempting connection to PSP.	Preventing indefinite gateway hanging.	Approving micro-transactions despite Risk Check outage.

The Idempotency Imperative

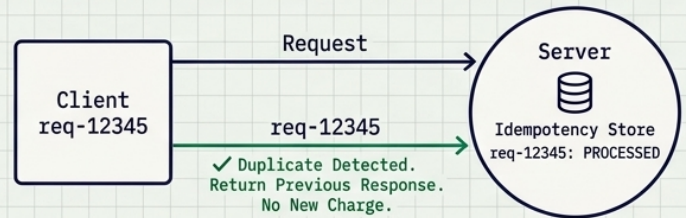
The Danger of Ambiguity



The Disaster: The client cannot know the true status. If they retry, they are charged twice.

The Solution: Idempotency Keys

Client generates a unique request ID. The server de-duplicates retried requests, ensuring multiple clicks produce exactly one charge.



Reconciling the Truth

Internal Immutable Ledger		
TXN-00123	2023-01-05 20:10:00	\$125
TXN-00124	2023-01-05 20:10:00	\$125
3 payments totaling \$225		
TXN-00125	2023-01-05 20:10:00	\$125
...	...	...



PSP Daily Settlement File (External)		
...	...	...
...	...	...
3 payments totaling \$225		
...	...	...
...	...	...

Mechanism: End-of-day batch process matches internal logs against external bank records. Any mismatch is instantly flagged to the Dispute Management service for investigation.

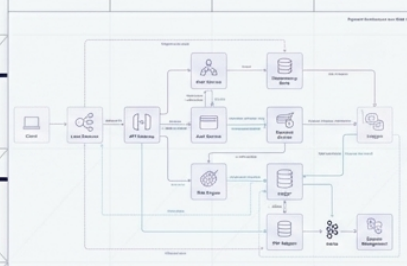
The Blueprint Realized

Phase 1: Requirements established the boundaries (Auth, APIs, Limits).

Phase 2: Scale math dictated the server count (782 servers) and database focus.

Phase 3: Incremental build mapped the baseline flow to the complex ledger.

Phase 4: Resilience rules ensured idempotency and guaranteed completion via Kafka.



Takeaway: Do not memorize the system. Internalize the framework. The structured process guarantees the architecture.